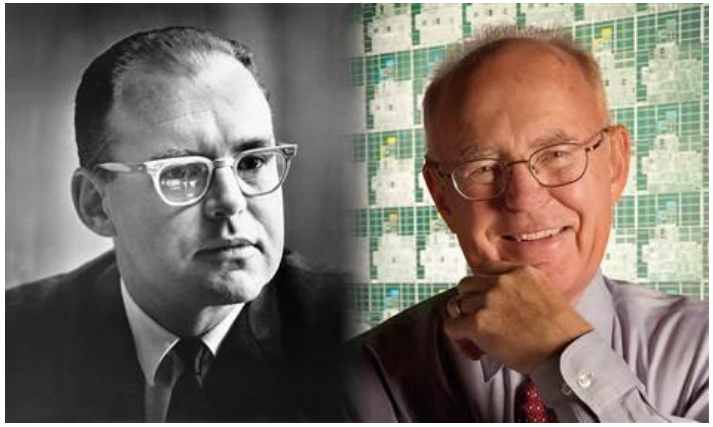
CONCURRENCY CONTROL

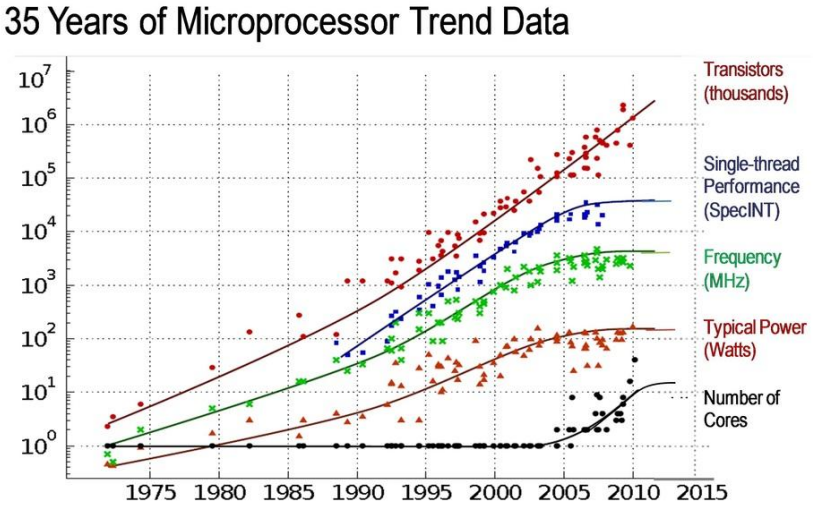
Shahram Ghandeharizadeh

Moore's Law

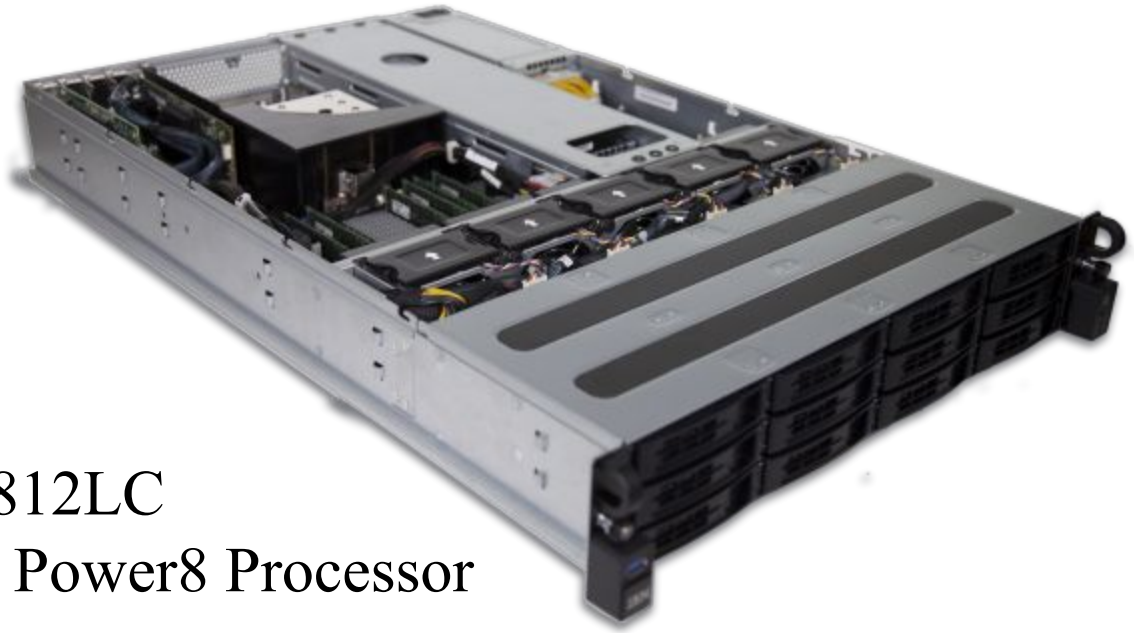


- In 1965, Moore observed:

Number of transistors per square inch on integrated circuits had doubled every year since their invention.



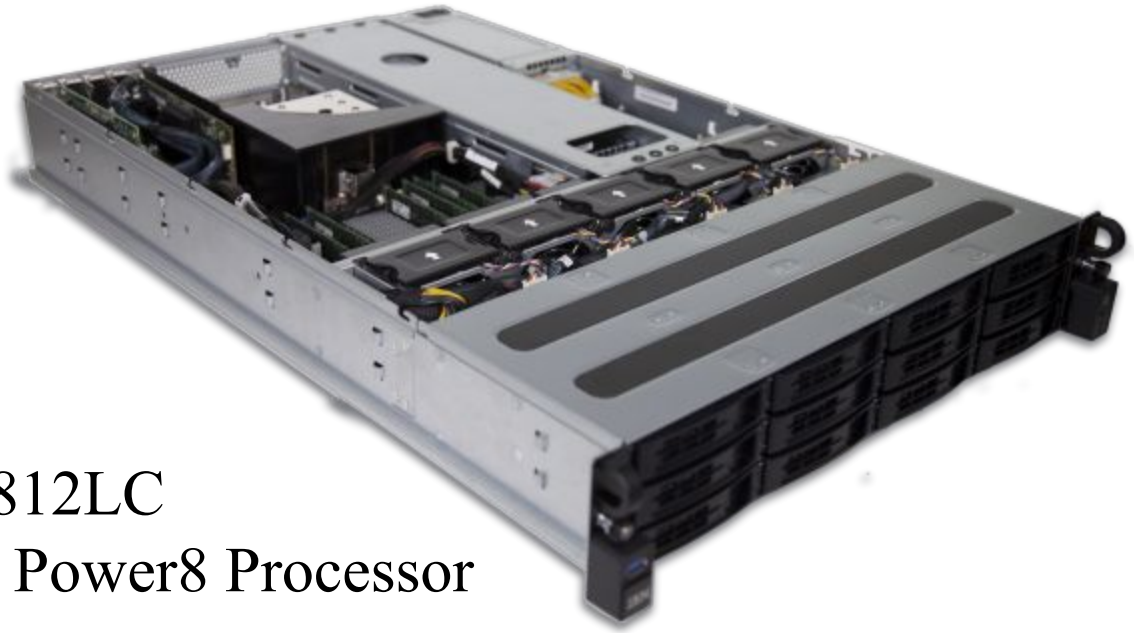
Motivation: Multi Core Servers



IBM Power System S812LC
One 8-core or 10-core Power8 Processor
1 TB of memory

Price Tag: ~5K

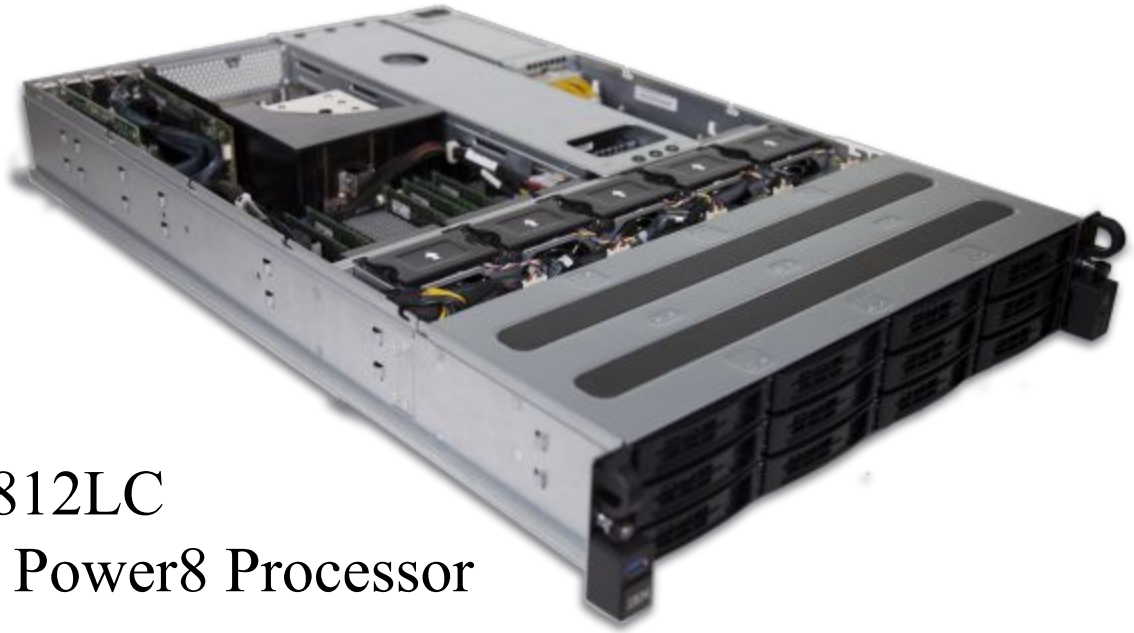
Multi Core Servers: Large Main Memory



IBM Power System S812LC
One 8-core or 10-core Power8 Processor
1 TB of memory

Inexpensive!
Most OLTP databases are 1 TB in size.

Multi Core Servers: Concurrent Transactions



IBM Power System S812LC
One 8-core or 10-core Power8 Processor
1 TB of memory

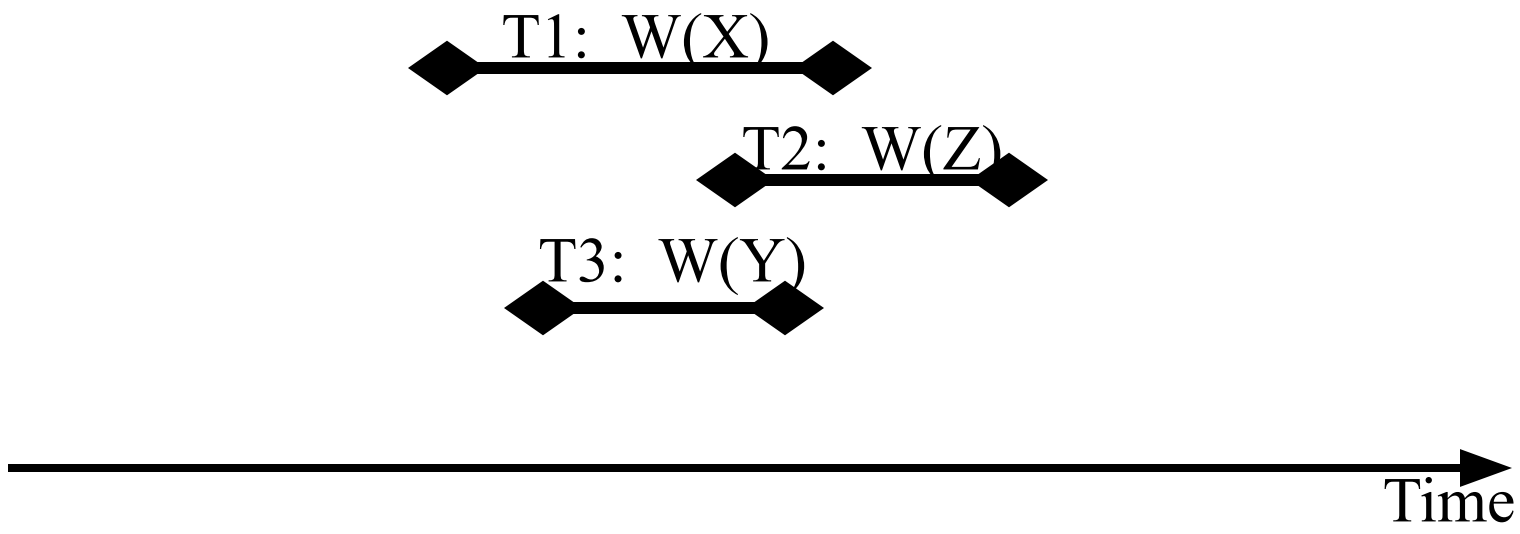
One transaction at a time execution utilizes one core, leaving other cores idle. Given C cores, utilization with one transaction at a time wastes (1-1/C) resources, e.g., 90% wasted with the 10-Core S812LC.

Concurrency Control Protocols

- Objective: Execute transactions concurrently using different cores while providing the illusion the transactions executed one at a time.
- How?
  - Serializable schedule: An execution of concurrent transactions such that the outcome (resulting database state) is equal to the outcome of transactions executing serially, without overlapping in time.

Serial Schedules

- Objective: Execute transactions concurrently using different cores while providing the illusion the transactions executed one at a time.
- How?
  - Serializable schedule: An execution of concurrent transactions such that the outcome (resulting database state) is equal to the outcome of transactions executing serially, without overlapping in time.



Serial Schedules

- Objective: Execute transactions concurrently using different cores while providing the illusion the transactions executed one at a time.
- How?
  - Serializable schedule: An execution of concurrent transactions such that the outcome (resulting database state) is equal to the outcome of transactions executing serially, without overlapping in time.

T1, T2, T3
T1, T3, T2
T2, T1, T3
T2, T3, T1
T3, T1, T2
T3, T2, T1

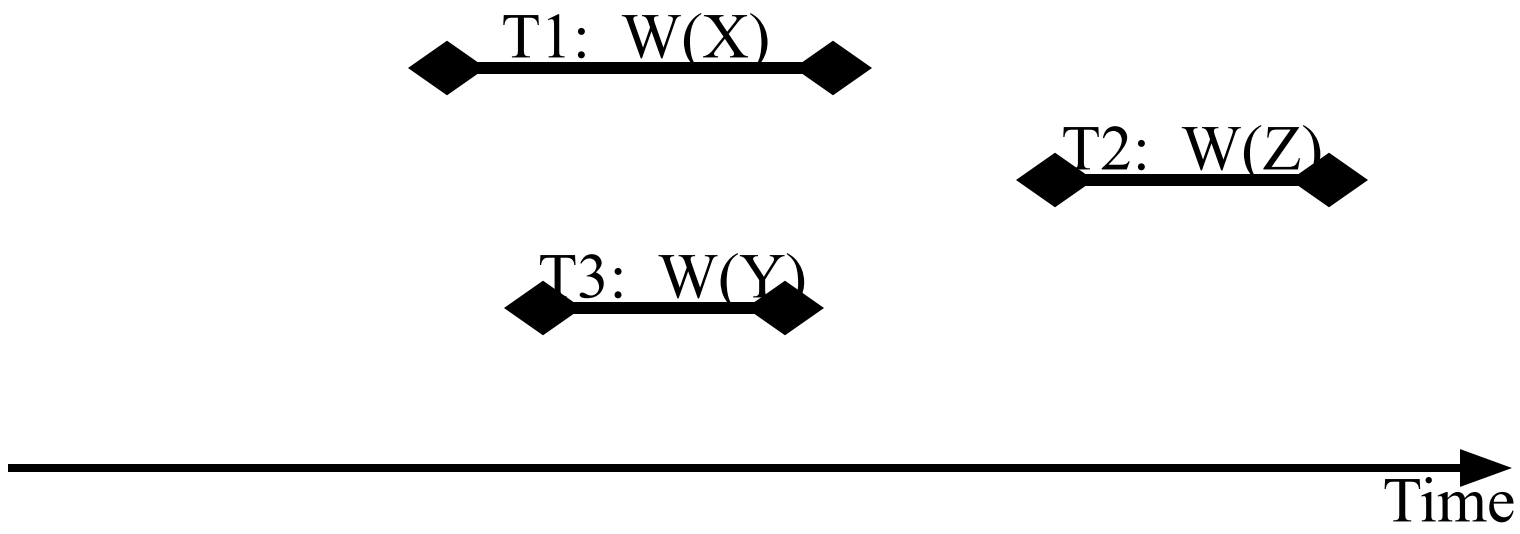
Serial Schedules

- Objective: Execute transactions concurrently using different cores while providing the illusion the transactions executed one at a time.
- How?
  - Serializable schedule: An execution of concurrent transactions such that the outcome (resulting database state) is equal to the outcome of transactions executing serially, without overlapping in time.

Given N transactions, there are N! serial schedules!

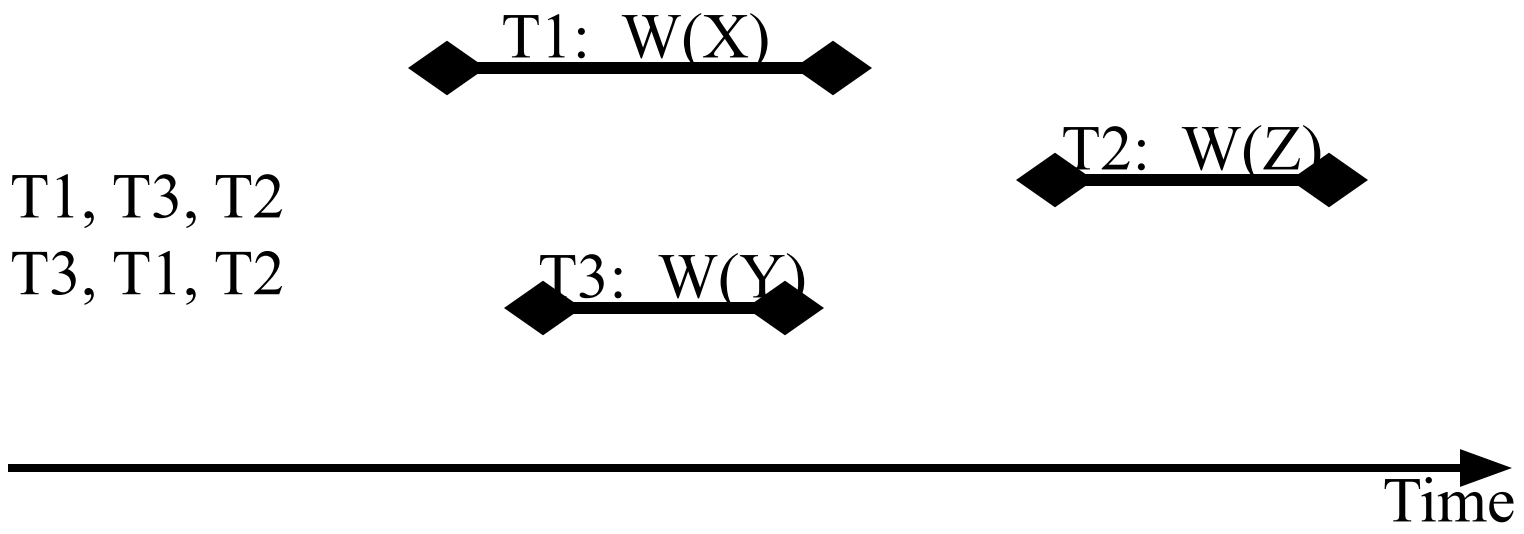
Strict Serial Schedules

- Objective: Execute transactions concurrently using different cores while providing the illusion the transactions executed one at a time.
- How?
  - Strict Serializable schedule: An execution of concurrent transactions such that the outcome (resulting database state) is equal to the outcome of transactions executing serially AND a transaction Tj that starts after the commit point of Ti is not re-ordered prior to it.



Strict Serial Schedules

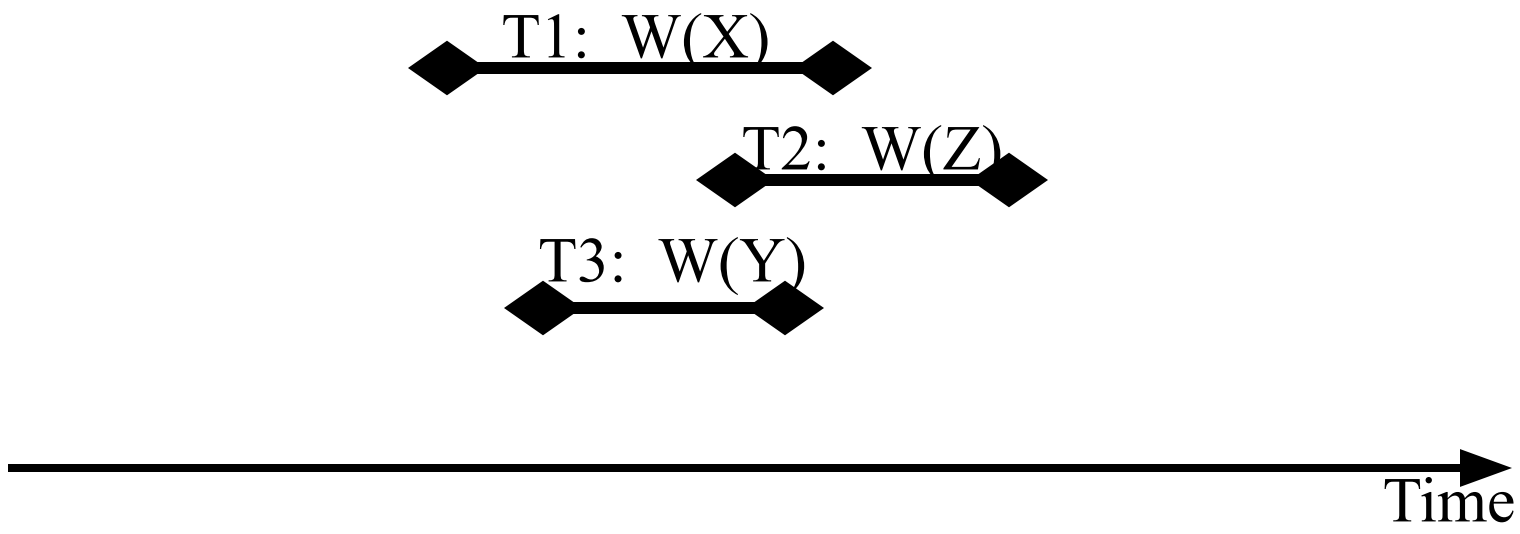
- Objective: Execute transactions concurrently using different cores while providing the illusion the transactions executed one at a time.
- How?
  - Strict Serializable schedule: An execution of concurrent transactions such that the outcome (resulting database state) is equal to the outcome of transactions executing serially **AND** a transaction Tj that starts after the commit point of Ti is not re-ordered prior to it.



Any Questions?

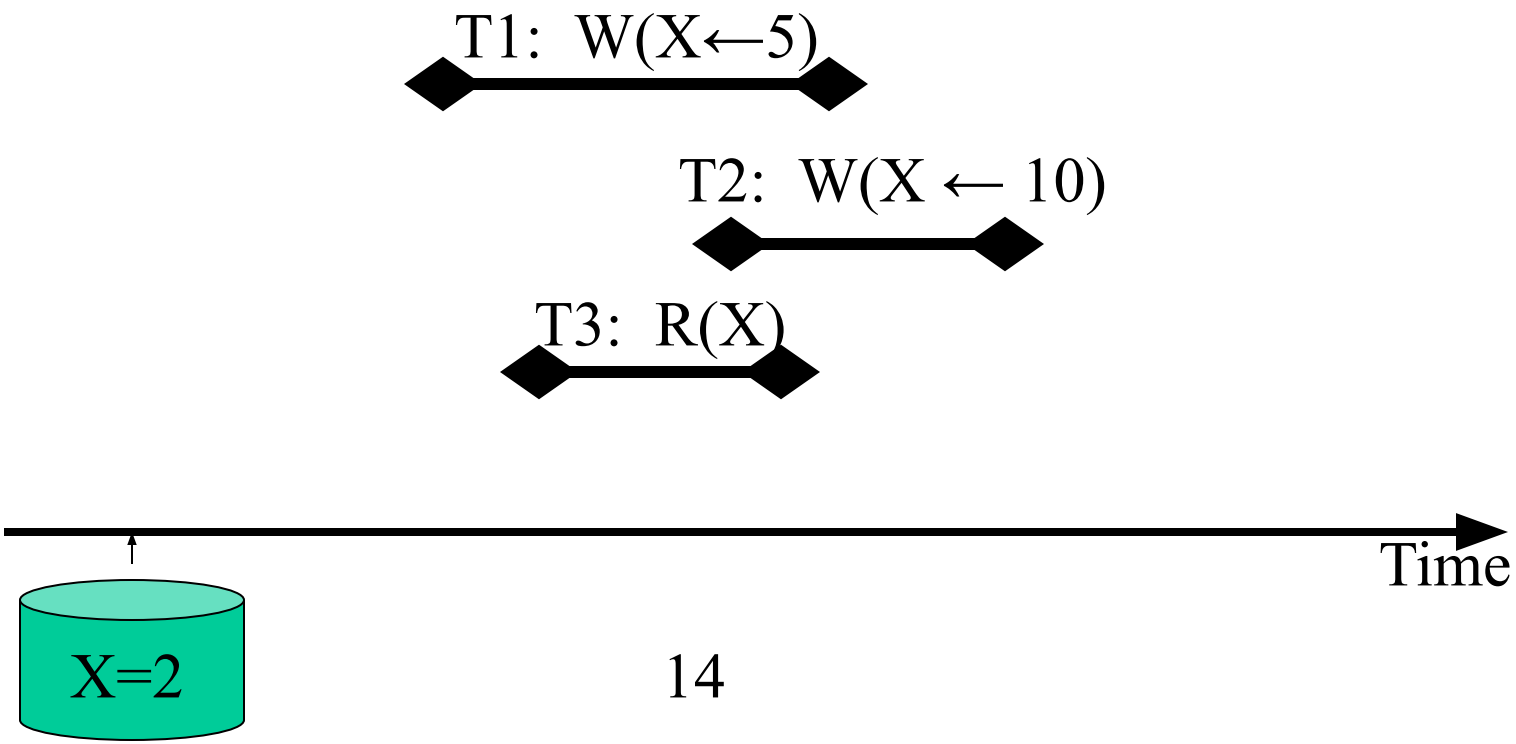
Any Questions?

- Is it important to compute a serial schedule for the following?



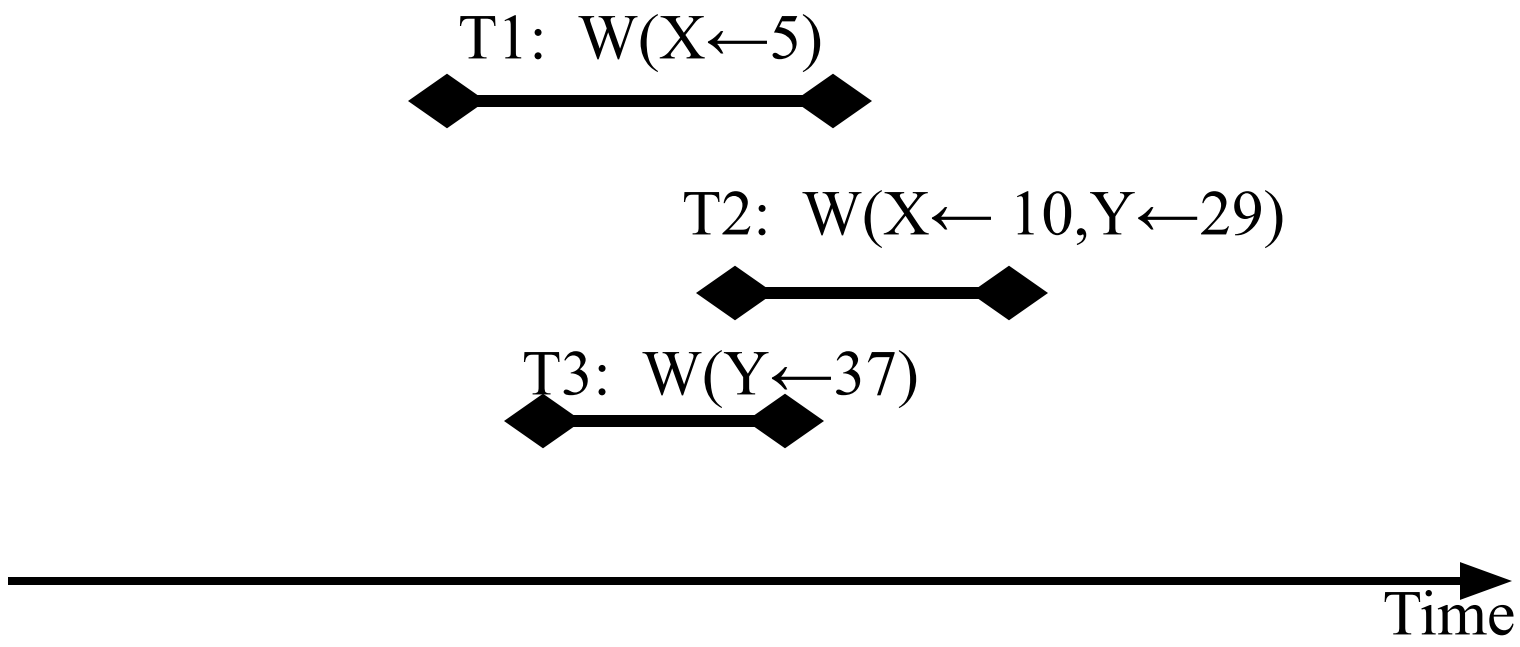
Any Questions?

- How about the following?
  - Assuming value of X is 2 at the start, what possible values may T3 observe?



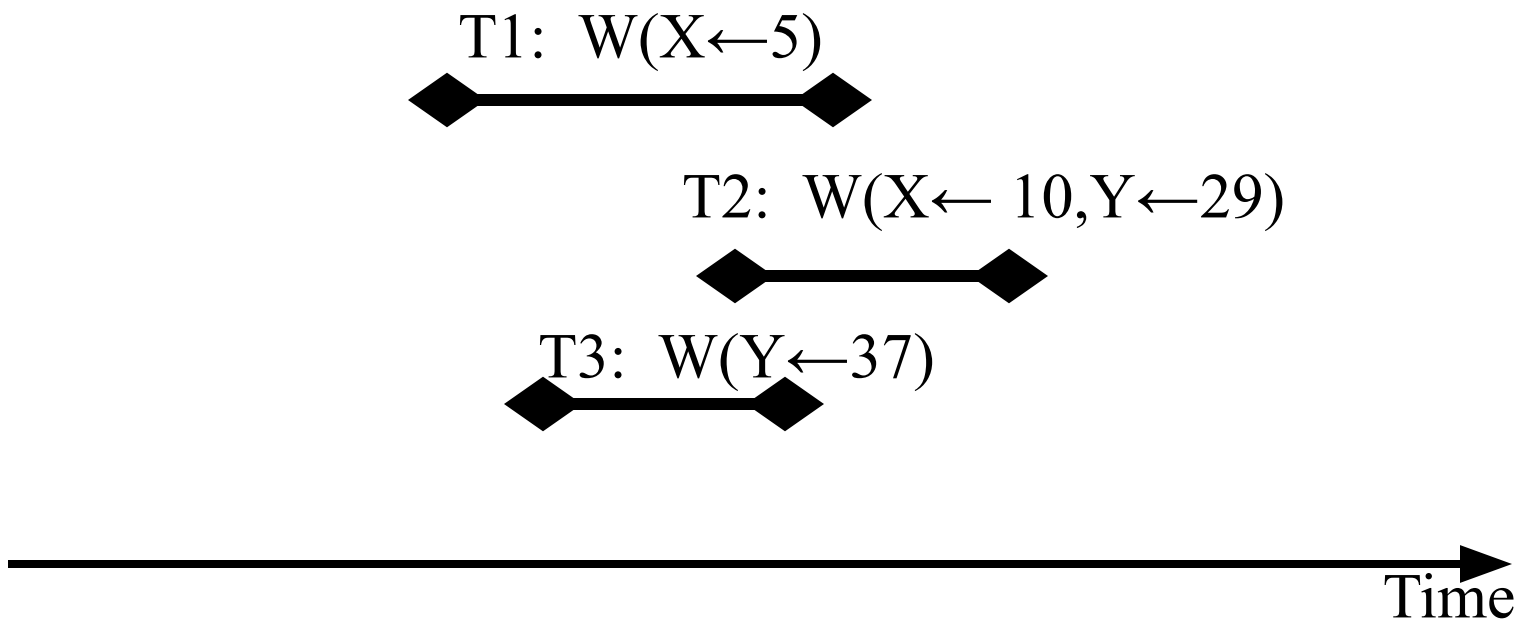
Any Questions?

- Consider the following 3 transactions overlapping in time.



Any Questions?

- 3! Schedules. However, how many are interesting?

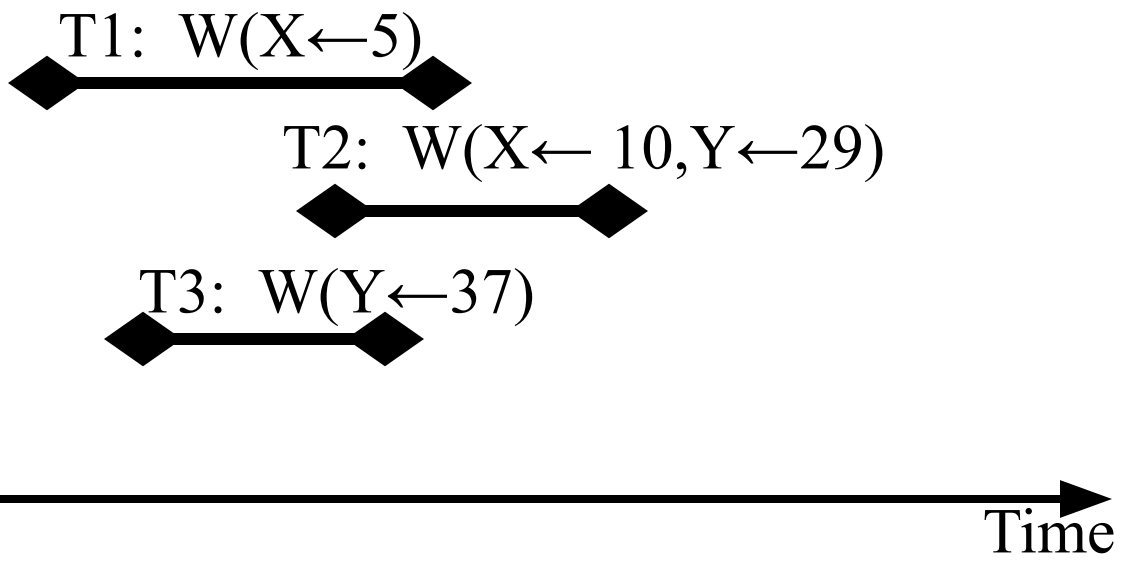


Any Questions?

- 3! Schedules. However, how many are interesting? 4 schedules

T1, T3, T2
T2, T1, T3
T1, T2, T3
T3, T2, T1

~~T3, T1, T2~~
~~T2, T3, T1~~



CONCURRENCY CONTROL INTRODUCTION

- Motivation: A dbms is multiprogrammed to increase the utilization of resources. While the CPU is processing one transaction, the disk can perform a write operation on behalf of another transaction. However, the interaction between multiple transactions must be controlled to ensure atomicity and consistency of the database.
- As an example, consider the following two transactions. T_0 transfers 50 from account A to B. T_1 transfers 10% of the balance from A to B.
  - T_0 T_1

    read(A) read(A)

    A=A-50 tmp = A × 0.1

    write(A) A = A - tmp

    read(B) write(A)

    B=B+50 read(B)

    write(B) B=B+tmp

    write(B)

CONCURRENCY CONTROL INTRODUCTION (Cont…)

- Assume the balance of A is 1000 and B is 2000. The bank's total balance is 3000. If the system executes T_0 before T_1 then A's balance will be 855 while B's balance will be 2145. On the other hand, if T_1 executes before T_0 then A's balance will be 850 and B's balance will be 2150. However, note that in both cases, the total balance of these two accounts is 3000.
- In this example both schedules are consistent.
- A **schedule** is a chronological execution order of multiple transactions by a system.
- A **serial schedule** is a sequence of processing multiple transactions which ensures atomicity of each transaction. Given *n* transactions, there are n! valid serial schedules.
- Concurrent execution of multiple transactions, where the instructions of different transactions are interleaved, may result in a non-serial schedule.
- To illustrate, assuming that a transaction makes a copy of each data item in order to manipulate it, consider the following execution:

CONCURRENCY CONTROL INTRODUCTION (Cont…)

T_0	T_1	
read(A)		
A=950	A=A-50	
	read(A)	A=1000
	tmp = A × 0.1	tmp=100
	A = A - tmp	
	write(A)	A=900
	read(B)	B=2000
A=950	write(A)	
B=2000	read(B)	
	B=B+50	
B=2050	write(B)	
	B=B+tmp	
	write(B)	B=2100

The final value of A and B are 950 and 2100, respectively. Note that the sum of these two values is 3050. This means that the bank has lost 50 due to the interleaved execution of instructions.

CONCURRENCY CONTROL INTRODUCTION (Cont…)

- In general, given two transactions T_i and T_j with instructions I_i and I_j :
  - If I_i and I_j refer to different data items then their execution order does not matter.
  - If they refer to the same data item Q then the order might be important:
    1. I_i = read(Q)
       I_j = read(Q)
       order does not matter
    2. I_i = read(Q)
       I_j = write(Q)
       order is important because:
       (1) if the order is I_i, I_j then T_i does not observe T_j's update
       (2) if the order is I_j, I_i then T_i observes the update of T_j

CONCURRENCY CONTROL INTRODUCTION (Cont…)

3. I_i = write(Q)
   I_j = read(Q)
   order is important because:
   (1) if the order is I_i, I_j then T_j observes T_i's update
   (2) if the order is I_j, I_i then T_j does not observe the update of T_i
4. I_i = write(Q)
   I_j = write(Q)
   order is important because the value obtained by the next read(Q) instruction observes either the value produced by I_i or I_j depending on whichever executed last.

LOCK-BASED PROTOCOLS

- There are alternative approaches to ensure serializability among multiple transactions.

Lock-based Protocols

- To ensure serializability, require access to data items to be performed in a mutually exclusive manner.
- This approach requires a transaction to lock a data item before accessing it. The simple protocol consists of two lock modes: **S**hared and e**X**clusive. A transaction locks a data item Q in S mode if it plans to read Q's value and X mode it it plans to write Q. The compatibility between these two lock modes is as follows:

	S	X
S	True	False
X	False	False

LOCK-BASED PROTOCOLS (Cont…)

- There can be multiple transactions with S locks on a particular data item. However, only one X lock is allowed on a data item. When a transaction requests a lock, if a compatible lock mode exists, it proceeds to lock the data item. Otherwise, it waits. Waiting might result in deadlocks.

LOCK-BASED PROTOCOLS (Cont…)

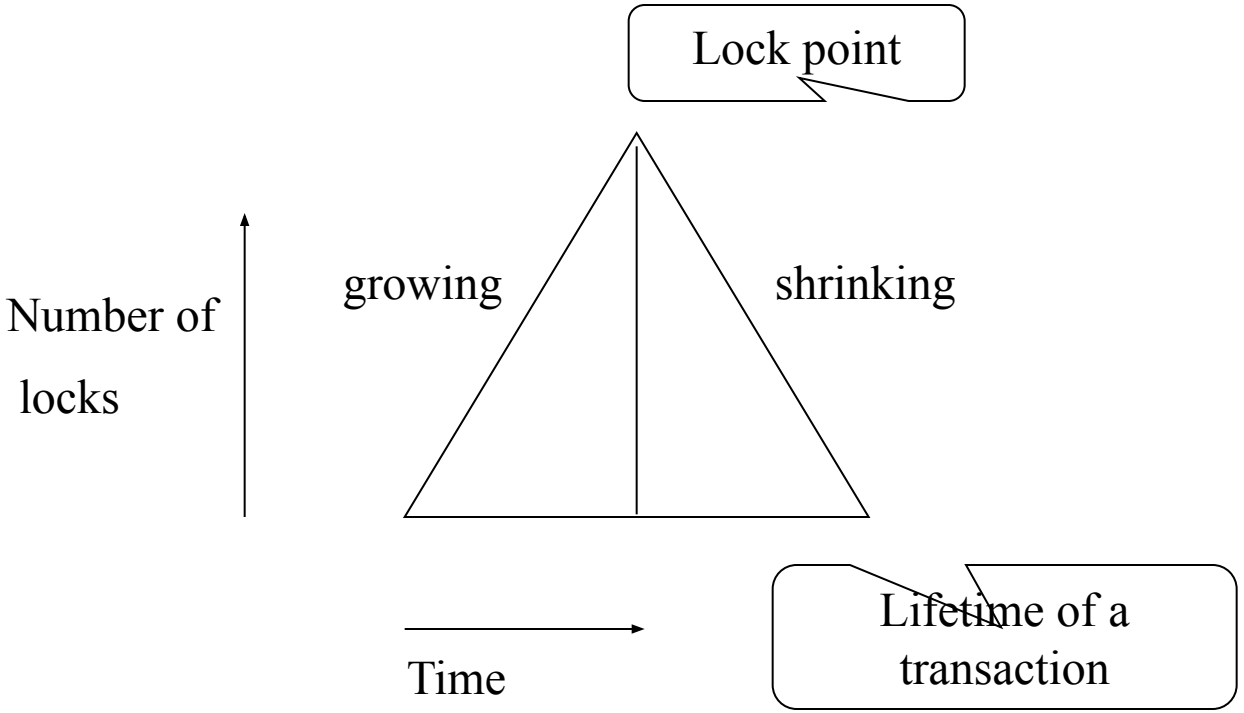
T_0	T_1	
lockX(A)		
read(A)		
A=A-50		
write(A)		
	lockX(B)	
	read(B)	
	tmp = B × 0.1	
	B = B - tmp	
	write(B)	
lockX(B)		
	lockX(A)	**DEADLOCK**
read(B)		
B=B+50		
write(B)		
	read(A)	
	A=A+tmp	
	write(A)	

LOCK-BASED PROTOCOLS (Cont…)

- As a solution to deadlocks, most systems construct a Transaction Wait for Graph (TWG). A transaction is represented as a node in TWG. When a transaction T_i waits for T_j, an arc is attached from T_i to T_j. Next, the system detects cycles in the TWG. If a cycle is detected then there exists a deadlock. The system breaks deadlocks by aborting one of the transactions (e.g., using log-based recovery protocol).

- With a two phase locking protocol, each transaction is required to release its locks at the end of its execution. Thus, a transaction has two phases:
  1. growing phase: the transaction acquires locks
  2. shrinking phase: the transaction releases locks and acquires no additional locks

LOCK-BASED PROTOCOLS (Cont…)



LOCK-BASED PROTOCOLS (Cont…)

Without a two-phase locking protocol, the schedule provided by an execution of transactions might no longer be serializable. This is specially true in the presence of aborts. Several possible situations might arise:

1. dirty reads: A transaction T_0 reads the value of a record Q at two different points in time (t_i and t_j) and observes a different value for this record. This is because an updating transaction T_1 produced the value of Q when T_0 read this value at time t_i. However, T_1 aborted sometimes later (prior to t_j) and when T_0 tried to read the value of Q at t_j, it observes the value of Q prior to execution of T_1.
2. un-repeatable reads: A transaction T_0 reads the value of a record Q at two different points in time (t_i and t_j) and observes a different value for this record. After T_0 reads the value of Q at time t_i, an updating transaction T_1 updates the value of Q and commits prior to t_j. When T_0 read this value of Q at time t_j, it observes a Different value for Q.

LOCK-BASED PROTOCOLS (Cont…)

Example of dirty reads:

time ↓

T_1	T_0
lockX(Q)	
read(Q)	
Q=Q+50	
write(Q)	
unlock(Q)	
	read(Q)
abort	
	read(Q)

Example of unrepeatable reads:

T_1	T_0
	lockS(Q)
	read(Q)
	unlock(Q)
lockX(Q)	
Q=Q+50	
write(Q)	
unlock(Q)	
commit	
	lockS(Q)
	read(Q)
	unlock(Q)

LOCK-BASED PROTOCOLS (Cont…)

3. dirty writes (lost updates): T_0 and T_1 read the value of Q at two different points in time and produce a new value for this data item. Subsequently, they overwrite each other when updating Q. The execution paradigm that motivated the use of locking (earlier in the lecture notes) is an example of dirty writes.

- Most systems support four levels of lock granularities:
  - Level 3: locks held to end of a transaction (two phase locking that results in serializable schedules)
  - Level 2: write locks held to end of a transaction (un-repeatable reads)
  - Level 1: no read locks at all (dirty reads and un-repeatable reads)
  - Level 0: no locks (dirty writes, dirty reads and un-repeatable reads)

MULTI-GRANULARITY LOCKING

- IS, IX, S, X, and SIX lock modes.

Optimistic Concurrency Control

Shahram Ghandeharizadeh
CSCI 585

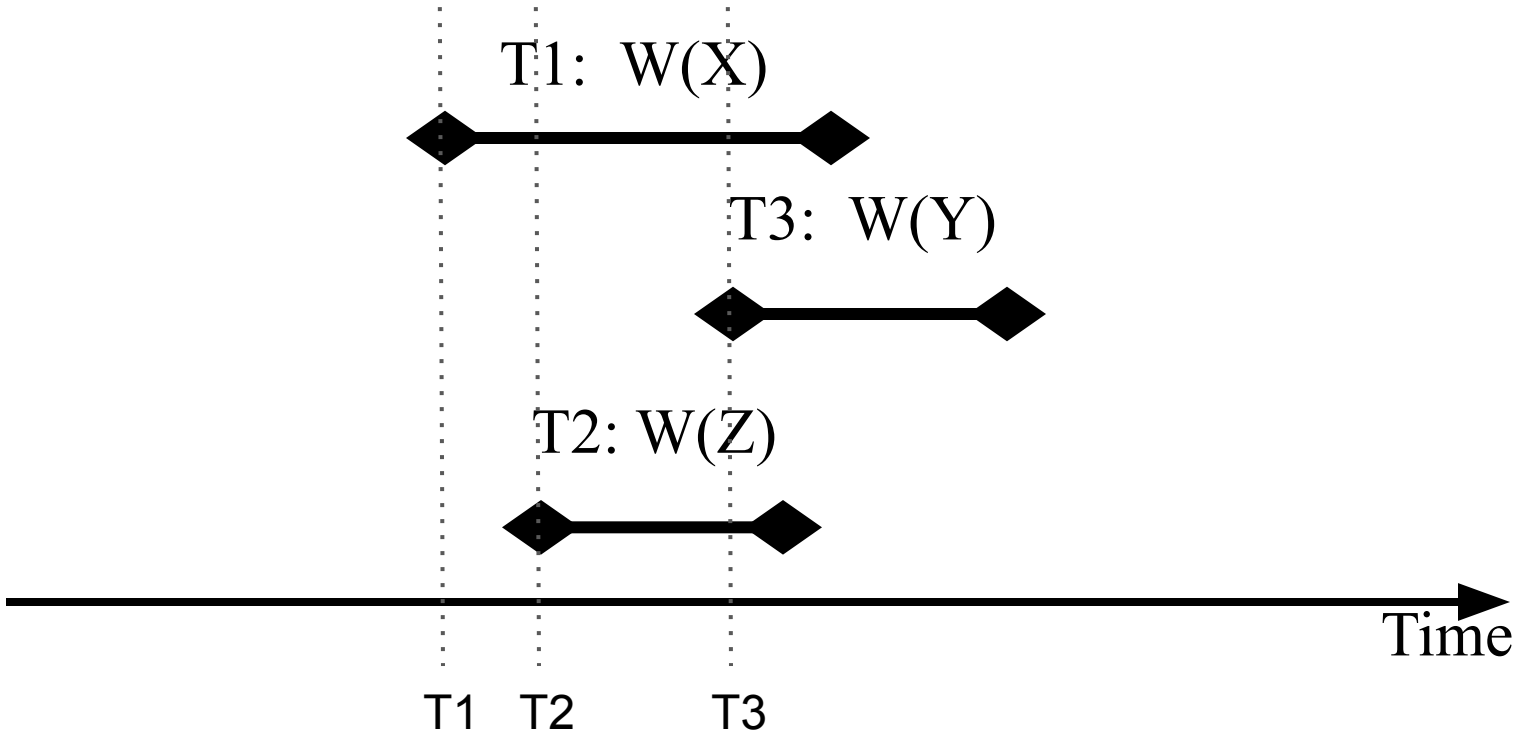
Time-stamp Based Protocols

Time-stamp based protocols provide a mechanism to enforce order. How?

- When a transaction T_i is submitted, we associate a unique fixed time stamp $TS(T_i)$. No two transactions may have an identical time stamp. One way to realize this is to use the system clock.
- The time stamp of the transaction determines the serializability order.
- Associated with each data item Q is two time stamp values:
  - W-TimeStamp(Q): Largest time stamp of the transaction that has written Q to date
  - R-TimeStamp(Q): Largest time stamp of the transaction that has read Q to dateT

Time-Stamp Based Protocols: Serial Schedules

- Objective: Execute transactions concurrently using different cores while providing the illusion the transactions executed one at a time.
- How?
  - Serial schedule is dictated by the arrival time of transactions.

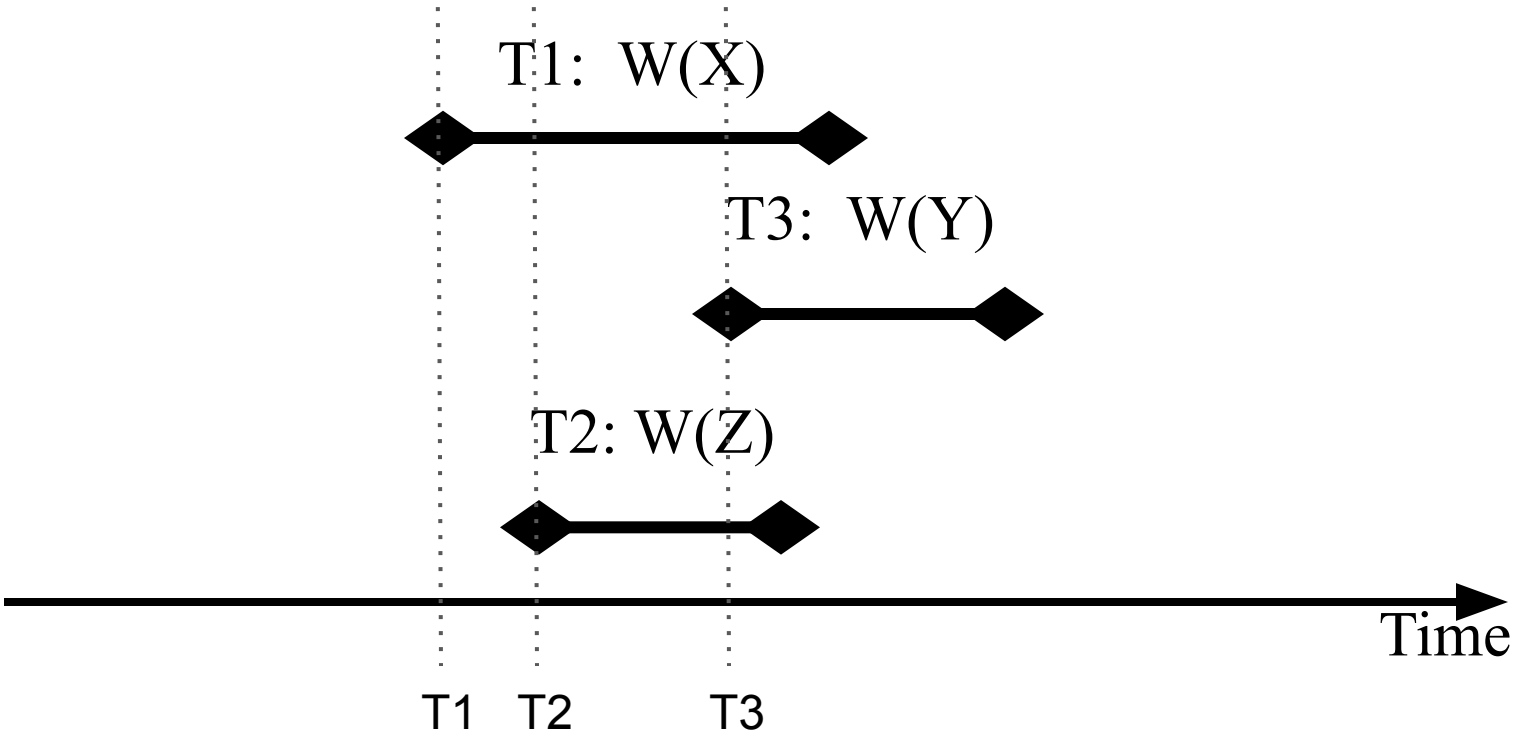


Time-Stamp Based Protocols: 1 Schedule Allowed

- Objective: Execute transactions concurrently using different cores while providing the illusion the transactions executed one at a time.
- How?

Only one possible schedule is allowed: T1, T2, T3

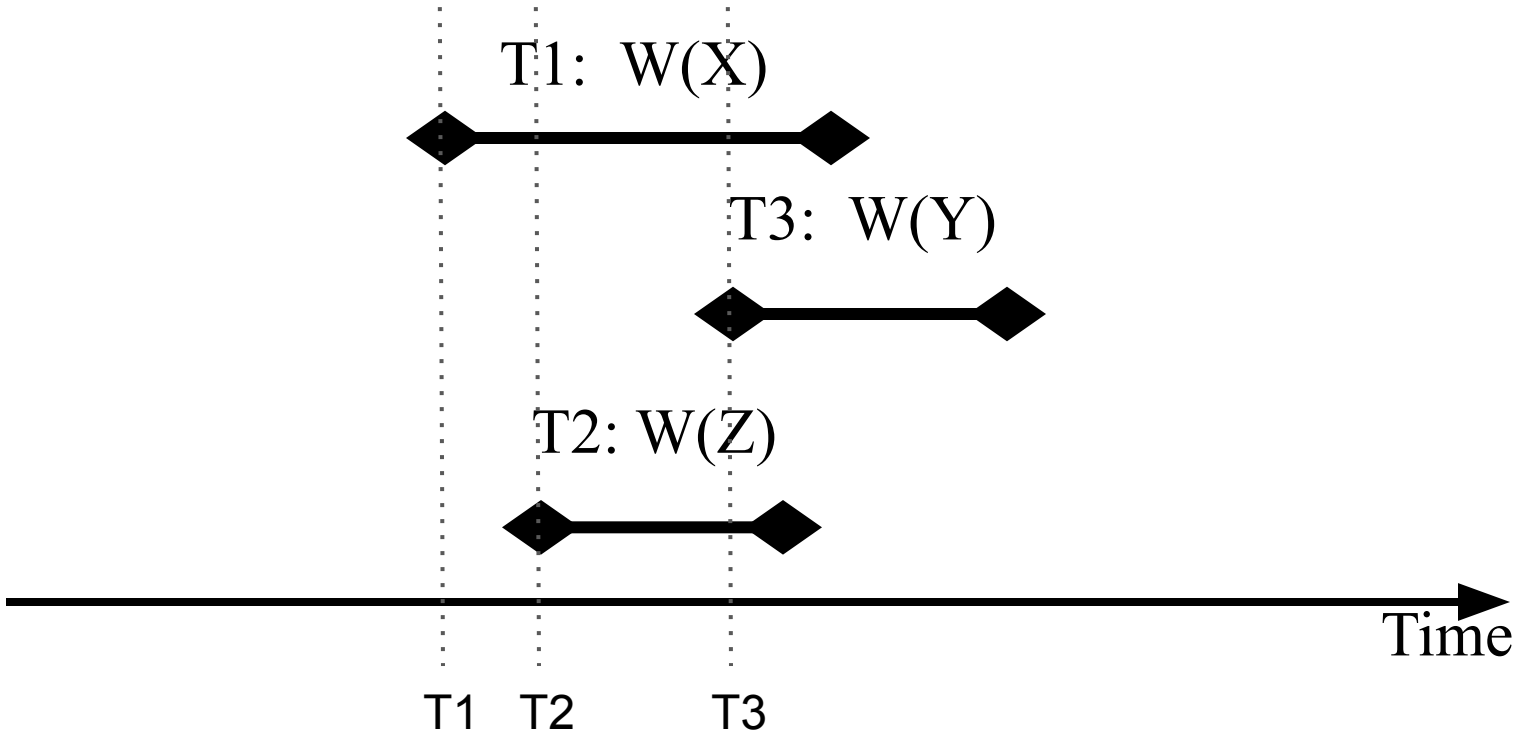
Questions?



Time-Stamp Based Protocols: Question?

- Objective: Execute transactions concurrently using different cores while providing the illusion the transactions executed one at a time.
- How?

Question: What does serial schedule T1, T2, T3 mean?

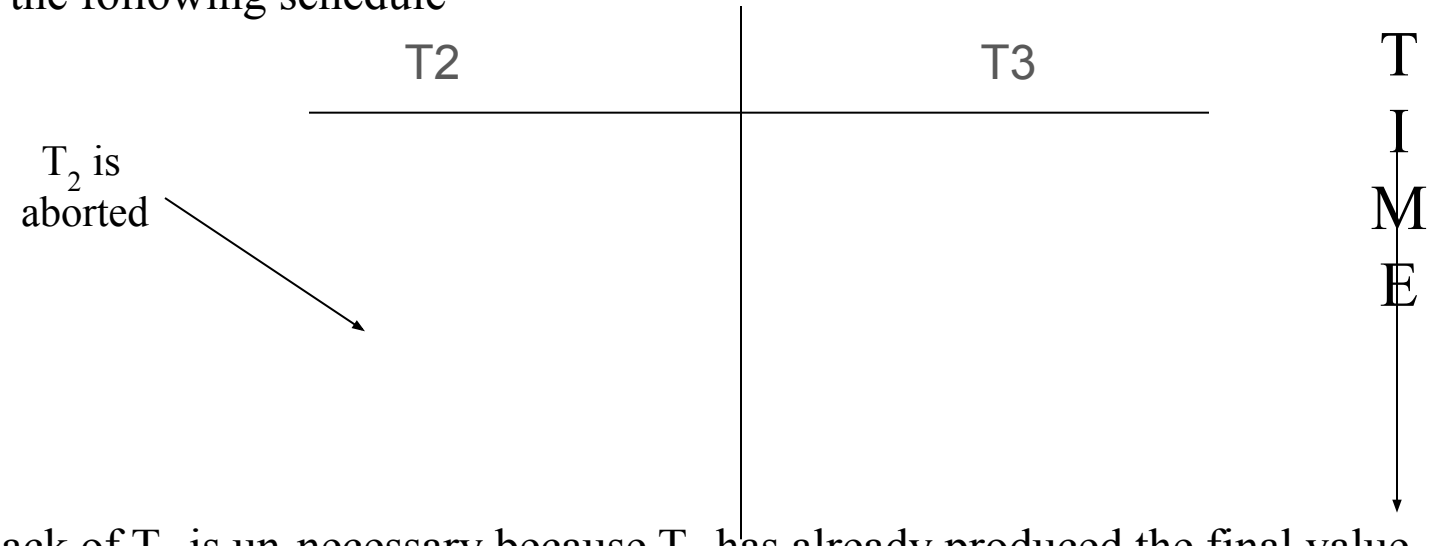


Time-Stamp Based Protocols: How?

- Let's assume that TS(T_i) is produced in an increasing order, i.e., $T_i < T_{i+1}$
- Suppose transaction T_i issues read(Q):
  - If **TS(T_i) < W-TimeStamp(Q)** then T_i needs to read the value of Q which was already overwritten. Hence the read request is rejected and T_i is rolled back.
  - If **TS(T_i) >= W-TimeStamp(Q)** then the read operation is executed and the R-timeStamp(Q) is set to the maximum of R-TimeStamp(Q) and TS(T_i).
- Suppose transaction T_i issues write(Q):
  - If **TS(T_i) < R-TimeStamp(Q)** then this implies that some transaction has already consumed the value of Q and T_i should have produced a value before that transaction read it. Thus, the write request is rejected and T_i is rolled back.
  - If **TS(T_i) < W-TimeStamp(Q)** then T_i is trying to write an obsolete value of Q. Hence reject T_i's request and roll it back.
  - Otherwise, execute the write(Q) operation and update **W-TimeStamp(Q)** to TS(Ti).
- When a transaction is rolled back, the system may assign a new timestamp to the transaction and restart its execution (as if it was just submitted).
- This approach is free from deadlocks.

Thomas's Write Rule

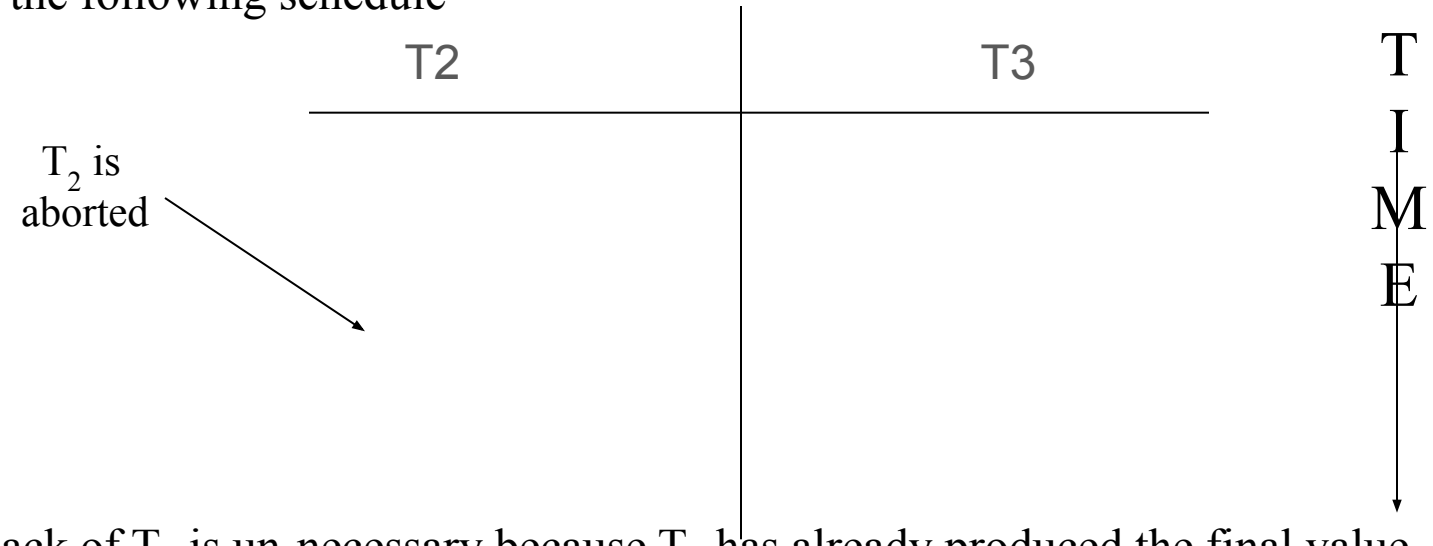
- Consider the following schedule



- The rollback of T_2 is un-necessary because T_3 has already produced the final value. The right thing to do is to ignore the write operation performed by T_2. To accomplish this, modify the protocol for the write operation as follows (the protocol for read stays the same as before): When T_i issues write(Q);
  - If **$TS(T_i) < R\text{-}TimeStamp(Q)$** then the value of Q that T_i is producing was previously read. Hence, reject the write operation and roll T_i back.
  - If **$TS(T_i) < W\text{-}TimeStamp(Q)$** then T_i is writing an obsolete value of Q. Ignore this write operation.
  - Otherwise, the write is executed, and W-TimeStamp(Q) is set to $TS(T_i)$

Thomas's Write Rule: Questions?

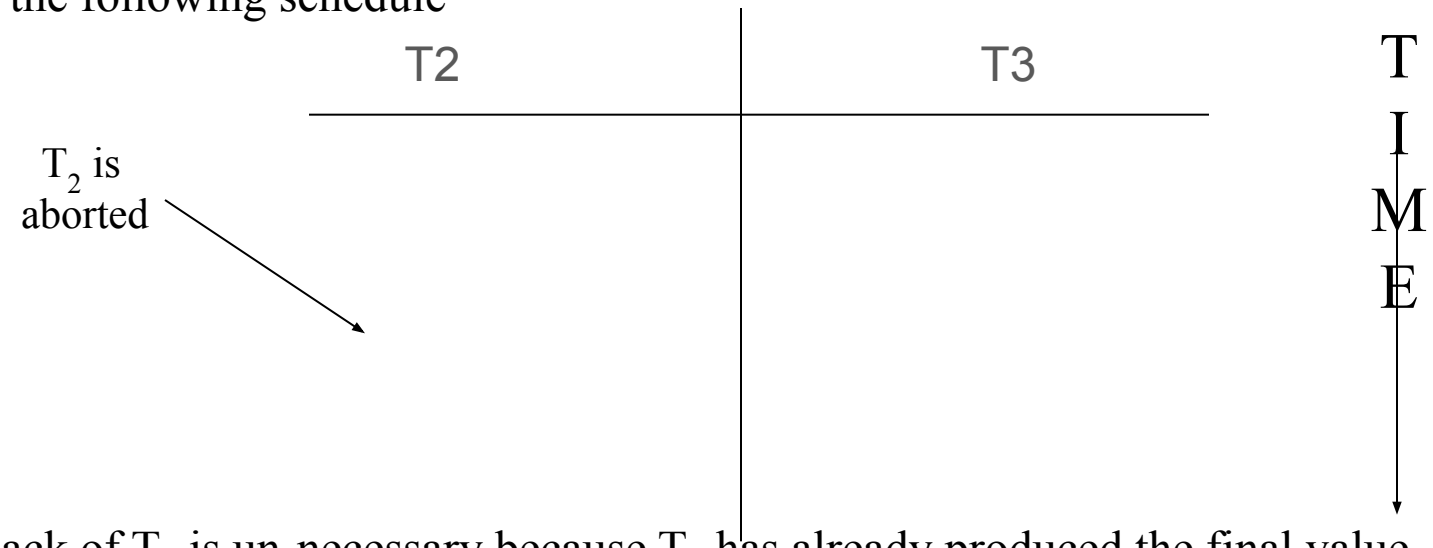
- Consider the following schedule



- The rollback of T_2 is un-necessary because T_3 has already produced the final value. The right thing to do is to ignore the write operation performed by T_2. To accomplish this, modify the protocol for the write operation as follows (the protocol for read stays the same as before): When T_i issues write(Q);
  - If **$TS(T_i) < R\text{-}TimeStamp(Q)$** then the value of Q that T_i is producing was previously read. Hence, reject the write operation and roll T_i back.
  - If **$TS(T_i) < W\text{-}TimeStamp(Q)$** then T_i is writing an obsolete value of Q. Ignore this write operation.
  - Otherwise, the write is executed, and W-TimeStamp(Q) is set to $TS(T_i)$

Thomas's Write Rule: Questions?

- Consider the following schedule



- The rollback of T_2 is un-necessary because T_3 has already produced the final value. The right thing to do is to ignore the write operation performed by T_2. To accomplish this, modify the protocol for the write operation as follows (the protocol for read stays

Question: What happens should T3 abort?

Hence, reject the write operation and roll T_i back.

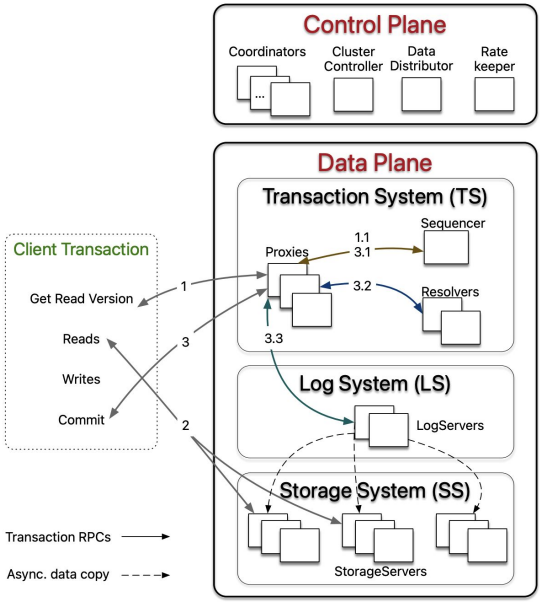
- If **$TS(T_i) < W\text{-}TimeStamp(Q)$** then T_i is writing an obsolete value of Q. Ignore this write operation.
- Otherwise, the write is executed, and W-TimeStamp(Q) is set to $TS(T_i)$

Optimistic CC (Validation Technique)

- Argues that the overhead of locking is too high and not worthwhile for applications whose workload consists of read-only transactions.
- Each transaction T_i has three phases:
  - Read phase: reads the value of data items and copies its contents to variables local to T_i. All writes are performed on the temporary local variables.
  - Validation phase: T_i determines whether the local variables whose values have been overwritten can be copied to the database. If not then abort. Otherwise, proceed to Write phase.
  - Write phase: The values stored in local variables overwrite the value of the data items in the database.
- A transaction has three time stamps:
  - Start(T_i): When T_i started its execution.
  - Validation(T_i): When T_i finished its read phase and started its validation
  - Finish(T_i): done with the write phase.

Optimistic CC (Validation Technique)

- Argues that the overhead of locking is too high and not worthwhile for applications whose workload consists of read-only transactions.
- Each transaction T_i has three phases:
  - **Read phase: reads the value of data items and copies its contents to variables local to T_i. All writes are performed on the temporary local variables.**
  - Validation phase: T_i determines whether the local variables whose values have been overwritten can be copied to the database. If not then abort. Otherwise, proceed to Write phase.
  - Write phase: The values st … the value of the data items in the database.
- A transaction has three time…
  - Start(T_i): When T_i started
  - Validation(T_i): When T_i fi… its validation
  - Finish(T_i): done with the v…



Optimistic CC: Validation Phase

- When validating transaction T_j, for all transactions T_i with $TS(T_i) < TS(T_j)$, one of the following must hold:
  - $Finish(T_i) < Start(T_j)$, OR
  - Set of data items written by T_i does not intersect with the set of data items read by T_j and T_i completes its write phase before T_j starts its validation phase.

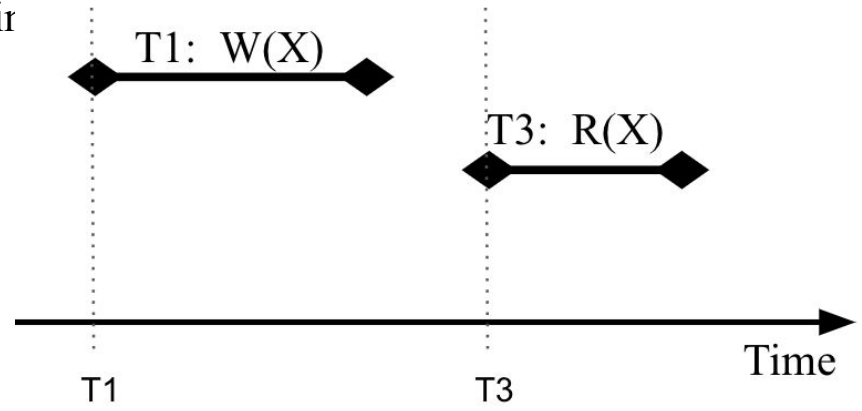
  - Rational: Serializability is maintained because the write of T_i cannot affect the read of T_j and since T_j cannot affect the read of T_i because:
    $Start(T_j) < Finish(T_i) < Validation(T_j)$

Optimistic CC: Validation Phase

- When validating transaction T_j, for all transactions T_i with $TS(T_i) < TS(T_j)$, one of the following must hold:
  - **$Finish(T_i) < Start(T_j)$**, OR
  - Set of data items written by T_i does not intersect with the set of data items read by T_j and T_i completes its write phase before T_j starts its validation phase.

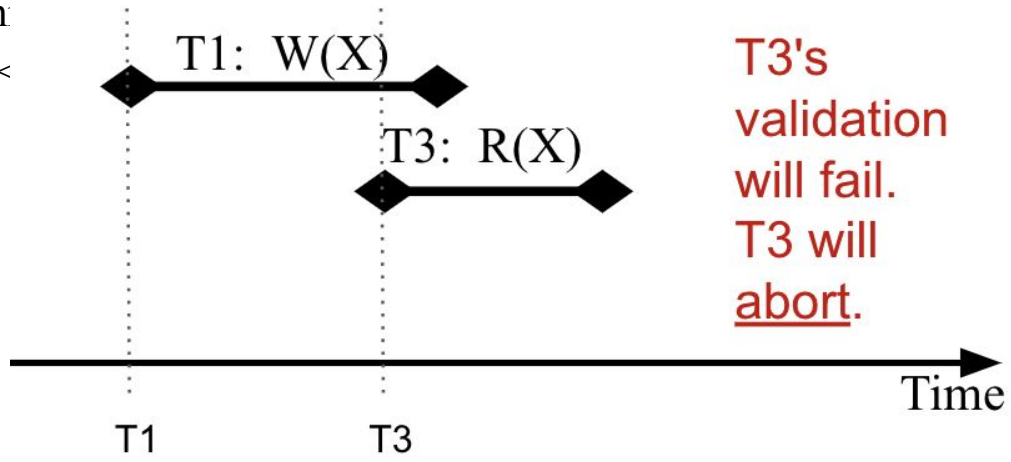
  - Rational: Serializability is maintained because the write of T_i cannot affect the read of T_j and since T_j cannot affect the read of T_i because:

Start(T_j) < Fir



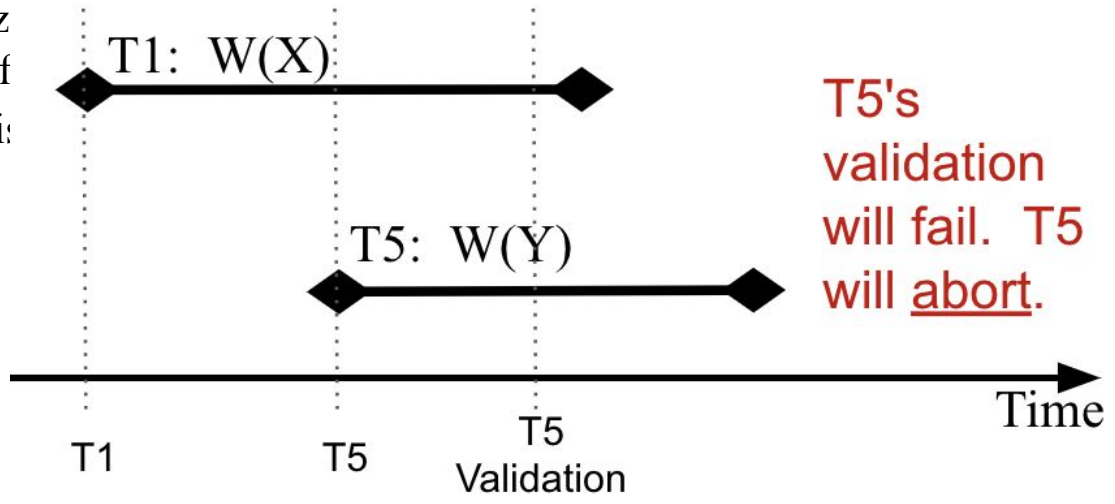
Optimistic CC: Validation Phase

- When validating transaction T_j, for all transactions T_i with $TS(T_i) < TS(T_j)$, one of the following must hold:
  - $Finish(T_i) < Start(T_j)$, OR
  - **Set of data items written by T_i does not intersect with the set of data items read by T_j** and T_i completes its write phase before T_j starts its validation phase.
  - Rational: Serializability is maintained because the write of T_i cannot affect the read of T_j and since T_j can
    $Start(T_j) <$



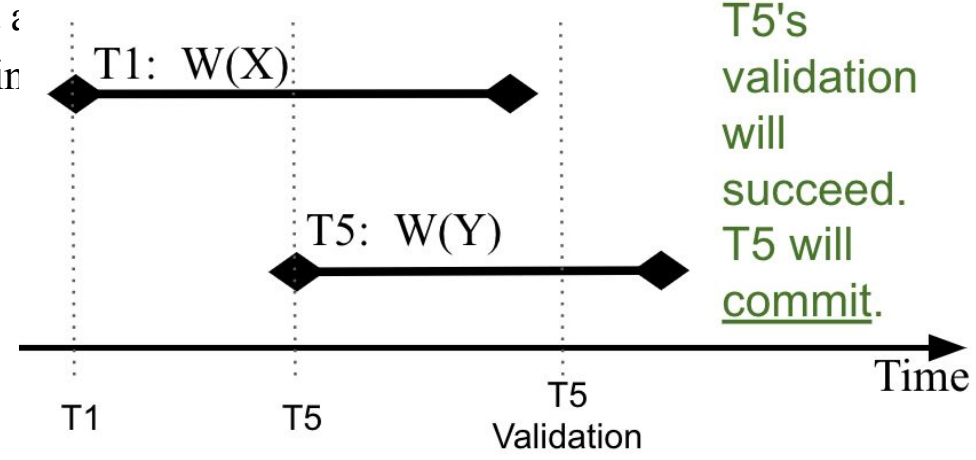
Optimistic CC: Validation Phase

- When validating transaction T_j, for all transactions T_i with $TS(T_i) < TS(T_j)$, one of the following must hold:
  - $Finish(T_i) < Start(T_j)$, OR
  - Set of data items written by T_i does not intersect with the set of data items read by T_j and **T_i completes its write phase before T_j starts its validation phase.**
  - Rational: Serializ… …d of T_j and since T_j cannot af… $Start(T_j) < Finis$…



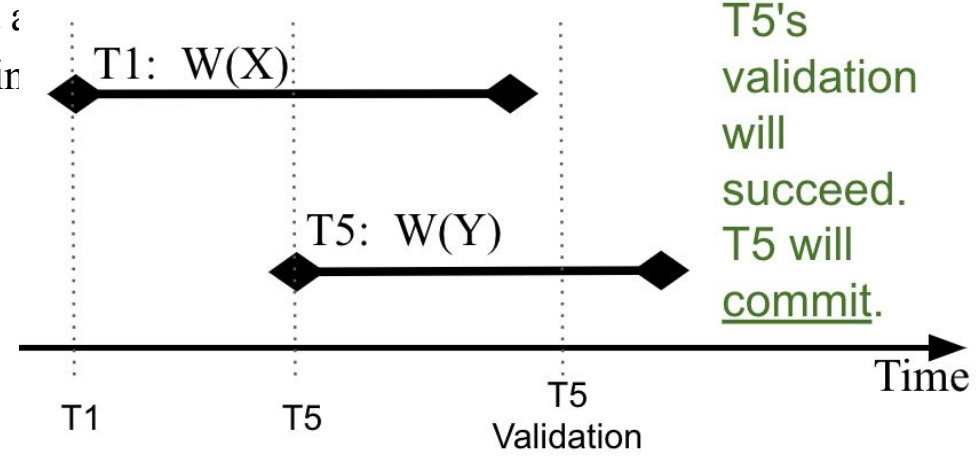
Optimistic CC: Validation Phase

- When validating transaction T_j, for all transactions T_i with $TS(T_i) < TS(T_j)$, one of the following must hold:
  - $Finish(T_i) < Start(T_j)$, OR
  - Set of data items written by T_i does not intersect with the set of data items read by T_j and **T_i completes its write phase before T_j starts its validation phase.**
  - Rational: Serializability is maintained because the write of T_i cannot affect the read of T_j and since T_j cannot a
    $Start(T_j) < Fir$



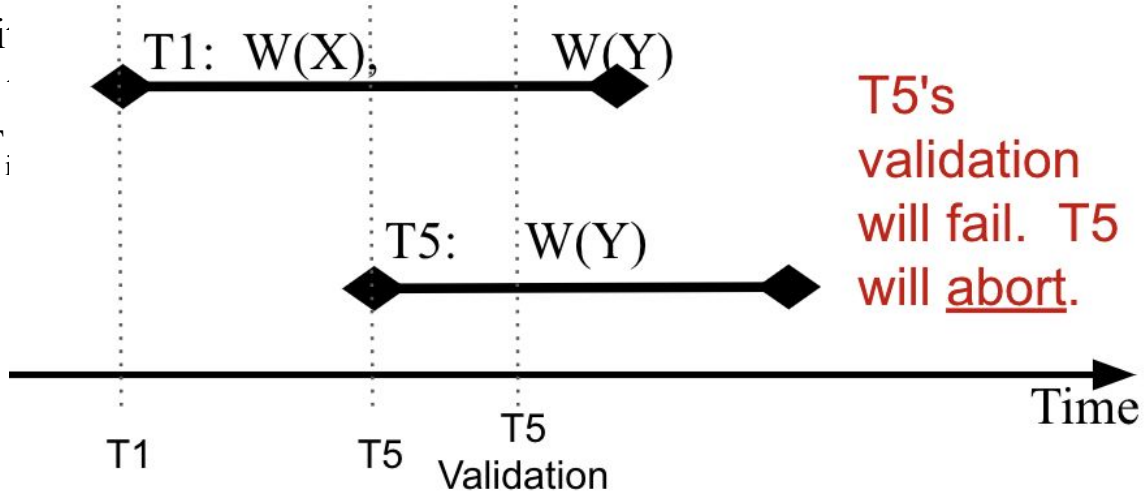
Optimistic CC: Validation Phase

- When validating transaction T_j, for all transactions T_i with $TS(T_i) < TS(T_j)$, one of the following must hold:
  - $Finish(T_i) < Start(T_j)$, OR
  - Set of data items written by T_i does not intersect with the set of data items read by T_j and **T_i completes its write phase before T_j starts its validation phase.**
  - Rational: Serializability is maintained because the write of T_i cannot affect the read of T_j and since T_j cannot a
    $Start(T_j) < Fir$



Optimistic CC: Validation Phase

- When validating transaction T_j, for all transactions T_i with $TS(T_i) < TS(T_j)$, one of the following must hold:
  - $Finish(T_i) < Start(T_j)$, OR
  - Set of data items written by T_i does not intersect with the set of data items read by T_j and **T_i completes its write phase before T_j starts its validation phase.**
  - Rational: Serializabili and since T_j cannot affect $Start(T_j) < Finish(T_i$



Multi-Version Schemes

- The system keeps track of the old version of a data item Q.
- When a Write(Q) operation is issued, the system creates a new version of Q
- When a Read(Q) operation is issued, the system **selects the right** version of Q to read
- As before, each transaction has a unique time stamp.
- A data item Q has a sequence of versions associated with it: Q_1, Q_2, ..., Q_n
- Each Q_k has three data fields:
  - Content: its value
  - W-TimeStamp: Timestamp of transaction that created version k
  - R-TimeStamp: Timestamp of the largest transaction that successfully read version k
- T_i creates a new version Q_k of data item when it issues Write(Q). Its content holds the new value. The W-TimeStamp and R-TimeStamp are initialized to $TS(T_i)$. R-TimeStamp value is updated whenever a transaction T_j reads the contents of Q_k and R-TimeStamp(Q_k) < $TS(T_j)$

Multi-Version Schemes: Reads and Writes

- Assume that transaction T_i issues either a read(Q) or a write(Q) operation
- Let Q_k denote the version of Q whose write timestamp is the largest write timestamp less than $TS(T_i)$, i.e., $W\text{-}TimeStamp(Q_k) < TS(T_i)$
- If T_i issues a Read(Q) then return the value of Q_k
- If T_i issues a write(Q), and $TS(T_i) < R\text{-}TimeStamp(Q_k)$ then T_i is rolled back
- Otherwise a new version of Q_k is created